Convoy: shipment tracking prototype

The cut I chose and why

I read the brief as two people with the same problem. A customer wants to know whether the order is on time and when it arrives. An ops person wants to know which shipments to chase first. Everything else was optional, so I built two roles and only the screens those two questions need.

The customer sees a shipment list, a detail page and a notification feed. Ops sees a Today page, an exceptions queue, a portfolio table and a detail page. The ops user is scoped to two of the four sites, matched on origin or destination. The scope lives in one selector, and one shipment sits outside it on purpose, so the rule is visible: its direct URL says "not found" and a search for it matches nothing.

What I left out, and why:

- A map. A good one costs hours and a cheap one looks generic. The leg-by-leg timeline carries the same information and became the centrepiece.
- Real-time transport and operator integrations. The brief is frontend only, so one scripted live event shows how freshness would feel.
- Document upload, analytics, dark mode and multiple languages. None of them changes the answer to the two questions above.
- Free-form LLM search. It needs a network and a key, and a canned parser lets me be honest about what is simulated.
- Automated tests. I chose to spend the hours on the product and checked it by hand: lint, build, both roles at 1440 and 360 px, a keyboard pass and reduced motion. That is a real gap, and I list it below.

Where and how the AI adds value

Six features, each on a screen where someone would need it. All are simulated with deterministic rules, and the interface says so by marking each one with a small "Convoy" stamp and a "Why" line that states what it was based on.

- Exception management. Ops Today and the exceptions queue rank shipments by a risk score built from delay, customs holds, silence, missing documents and port incidents. Each row proposes one next action: draft a notice, request a document or contact the operator.
- Arrival estimates. A shipment detail shows an estimate with a confidence word and a reason, for example the vessel delay at Tarragona plus typical customs time at Gdansk.
- Consolidation. Six operators speak six vocabularies: "Gate-in", "VSL DEP", "Docs lodged", "Dédouané". Convoy maps them to one set of milestones and shows one timeline. Ops can see the raw feed beside the normalised one.
- Daily summary. Today opens with a sentence built from live counts, followed by ranked next actions. It stays true after ops act, because it is recomputed from the same facts.
- Natural-language search. In the portfolio, "shipments to France this week running late" becomes three visible chips and a filtered table. Words it cannot use are shown as ignored.
- Proactive notices. From the queue, ops opens a drafted message in plain language, edits it and sends it. The customer then sees it as a banner on the shipment and in the feed. This is the one

place the two roles meet.

The principle behind all of it is that a prediction must never pass as a fact. An operator event with a timestamp is confirmed and drawn solid. Anything else, planned or estimated, is dashed or hatched, and a single `Estimate` component is the only code allowed to render it. If nothing moved the date, the interface says "Planned", not "Estimated". An unknown operator status is shown as "Not mapped" and kept raw. I would rather show a gap than guess.

Technical approach

React 19 with Vite, react-router for real URLs, plain CSS with custom-property tokens, Barlow as the only type family and lucide for icons. No backend, no UI kit, no state library. This was the fastest route to something polished, because I control every pixel and there is nothing to fight.

The central decision is to store facts and derive judgments. The store, a context with a reducer, holds operator events, the plan, documents and what ops did. Risk, arrival estimates, the summary, search results and drafted notices are pure functions in `src/ai/` run through selectors. That makes the reliability rule structural, since the UI cannot display an estimate as confirmed. It also means ops actions flow to the customer view with no extra wiring, and the folder is a clean line to point at: this is what production would replace.

The risk rules are plain weights, such as +4 for a customs hold, +3 for 24 hours of silence and +2 per day of slip, capped. Same inputs give the same ranking. The dataset has 24 shipments, six operators and a fixed "now" of 8 Oct 2026, 14:30, so the demo renders the same on any machine.

For production, the AI path would look like this:

- Operator statuses. An LLM would extract and map free-text statuses to milestones, validated against a schema. A rules table stays as the fallback, and anything below a confidence threshold lands in "Not mapped", as it does here.
- Arrival estimates. A model trained on lane history, with vessel and port feeds as inputs, would return an estimate and a confidence. Thresholds decide whether the customer sees a date, a range or "no update yet".
- Notices. Drafts come from the model, but a human approves every message that reaches a customer.
- Search. The LLM would turn text into the same filter object the dropdowns use, so the chips stay inspectable and a bad parse can be corrected.
- Evaluation and freshness. A labelled set of real operator statuses and past delays would score each change before it ships. Operators push events by webhook where they can, and polling covers the rest.

Assumptions, limitations and next steps

I assumed one customer and one ops persona are enough to show scoping, that English, metric units and EUR are fine, and that operators expose events with timestamps. I also assumed the demo runs locally, so there is no deployment or auth.

Limitations I know about:

- Reloading resets all data.
- Search understands a fixed set of phrases and nothing else.

- Risk and estimates are deterministic rules, not models, so they cannot learn.
- There is only one scripted live event.
- No auth, no tests, and a few rough edges: in ops, a port drop reads "Delivered" in the normalised column but "Delivered to port" on the timeline, and I added an "Awaiting update" customer status for stale shipments that was not in the spec.

Next steps, in order. First, real events behind a thin API with webhooks. Second, an evaluation set for the status mapper, before any LLM touches customer-facing text. Third, tests for the risk and search rules, which are the cheapest to cover. Then notification preferences, document upload and a map, once I know what customers actually ask for.

I used AI coding tools (Claude Code) throughout. I made the scope, product and visual decisions and reviewed what it produced.